

计算机程序设计基础(2)

--- C++程序设计(9)

孙甲松

sunjiasong@tsinghua.edu.cn

电子工程系 信息认知与智能系统研究所

罗姆楼6-104

电话: 62796193/13901216180

2020. 4.

7. 异常处理

- 在程序执行的过程中若出现一些不寻常的情况，或运行结果出现无法定义的情况，而使得程序操作（执行）不得不断被中断时，我们就说程序出现了异常。
- C++中的异常控制结构的功能是，当函数出现异常，函数的执行被终止，使控制权从函数返回，返回点是调用函数所指定的一个地点，而不是调用发生的地点。
- 这么做的主要目的是为了容错，提高程序的鲁棒性（robustness）。程序提前自预测可能的出错，（被零除，内存申请不成功、所指定的文件打不开，等等），打印出错信息并正常退出，将控制权交给上一级的类或程序处理该异常，而不让程序出现致命错误，导致出现死机等无可挽回的情况。

- ◆ 语法形式是：

 抛出异常：

throw 表达式；

 捕获异常：

try { 函数调用 }

catch (声明) 语句

- ◆ 被调用函数中使用**throw**，当满足条件由**throw**抛出异常，而由**try**捕捉其后面{ }中函数调用出现的异常，**throw**的表达式对应**catch**的声明。一个**try**块后可以有多个**catch**分别处理不同的**throw**异常。
- ◆ 若程序中有多处要抛出各种不同的异常，应该用不同的（表达式）操作数类型来相互区别，操作数的值不能用来区别不同的异常。

- ◆ C++的异常处理机制使得异常的引发和处理不必在同一个函数中，可以将异常处理集中在一个专门的函数中进行。使得程序能从异常事件中恢复过来。

例如：

```
#include<iostream>
using namespace std;
double Div(double a, double b)
{ if (b == 0.0) throw 1.0E-308;
  return a/b;
}
int Div(int a, int b)
{ if (b == 0) throw 0;
  return a/b;
}
```

```

void main( )
{ try {
    cout << "5.2/2.0=" << Div(5.2, 2.0) << endl;
    cout << "5/2=" << Div(5, 2) << endl;
    cout << "5.2/0.0=" << Div(5.2, 0.0) << endl;
    cout << "5.2/4.0=" << Div(5.2, 4.0) << endl;
    cout << "5/0=" << Div(5, 0) << endl;
    cout << "5/4=" << Div(5, 4) << endl;
}
catch (double d)
{   cout << "Exception, float overflow, why->" << d << endl;}
catch (int i)
{   cout << "Exception, integer overflow, why->" << i << endl;}
cout << "All done!" << endl;
}

```

7. 异常处理(2)

运行结果:

$5.2/2.0=2.6$

$5/2=2$

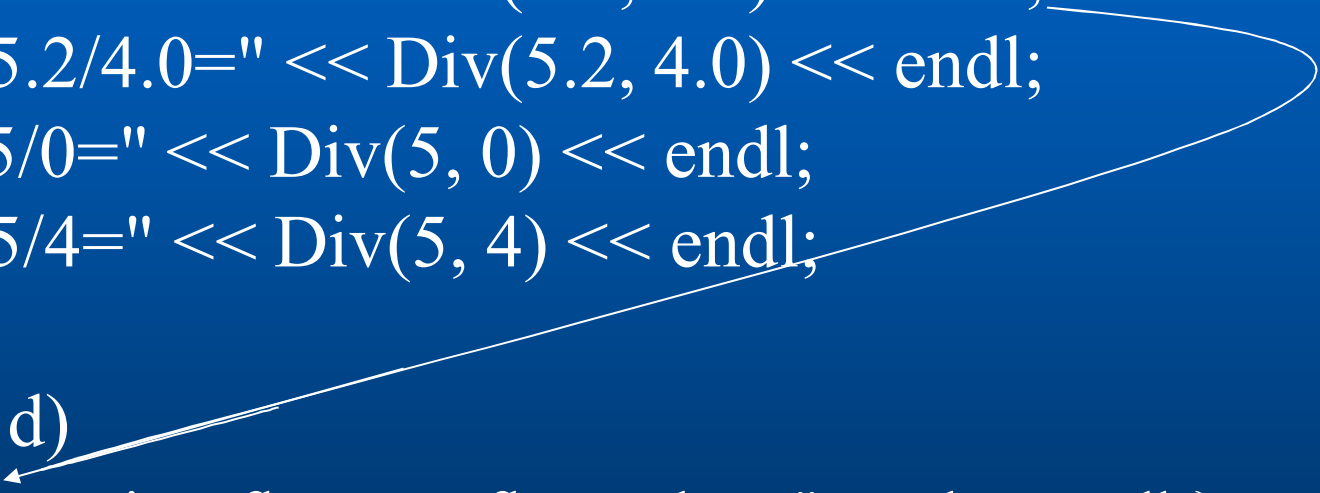
Exception, float overflow, why->1e-308

All done!

请按任意键继续...

- ◆ 由于第3个Div函数调用出现异常，throw抛出一个最小double数1e-308，转向catch处理。被第一个catch捕获，打印出错误信息后，正常结束了程序的执行。第4个Div函数及其以后的函数调用语句并没有被执行。

```
void main()
{ try {
    cout << "5.2/2.0=" << Div(5.2, 2.0) << endl;
    cout << "5/2=" << Div(5, 2) << endl;
    cout << "5.2/0.0=" << Div(5.2, 0.0) << endl;
    cout << "5.2/4.0=" << Div(5.2, 4.0) << endl;
    cout << "5/0=" << Div(5, 0) << endl;
    cout << "5/4=" << Div(5, 4) << endl;
}
catch (double d)
{   cout << "Exception, float overflow, why->" << d << endl;}
catch (int i)
{   cout << "Exception, integer overflow, why->" << i << endl;}
cout << "All done!" << endl;
}
```



异常处理相当于在C语言程序中的某种判断终止程序运行:

```
char *p;  
p = (char *)malloc(10);  
if (p == NULL)  
{   printf("Can't get memory!\n");  
    exit(0);  
}
```

当未能申请到内存, p为空, 此时打印出:

Can't get memory!

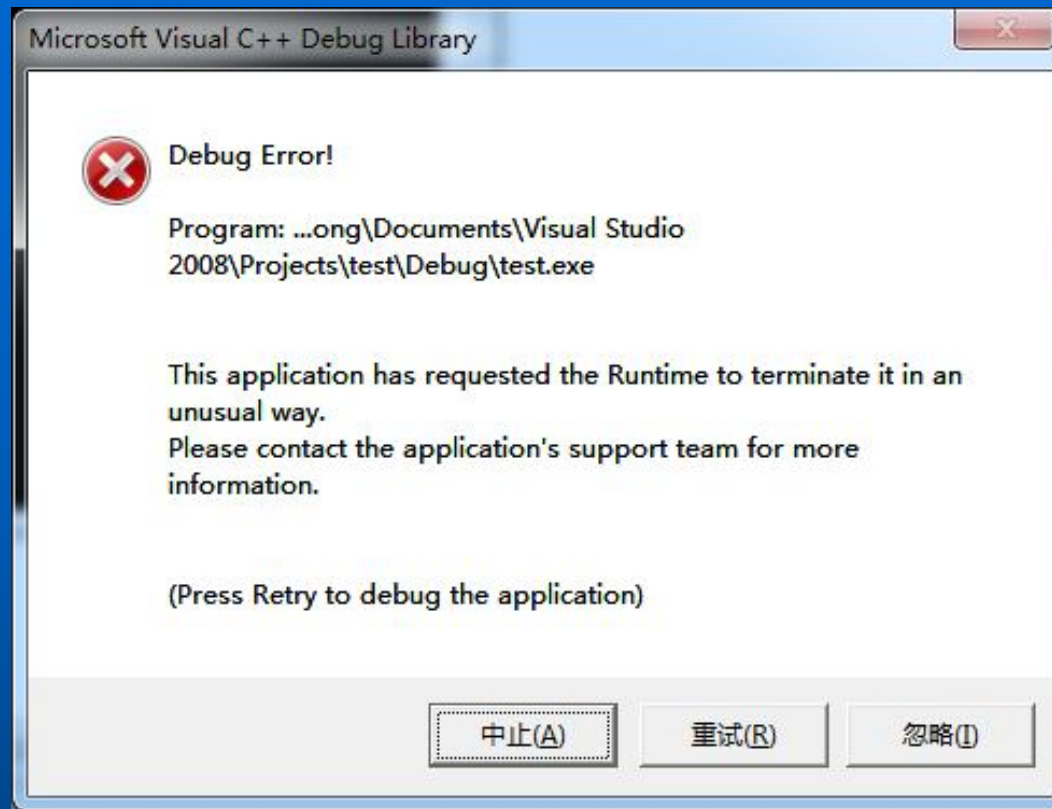
后执行exit(0)将强行终止整个程序的运行, 无论此程序的后面是否还有语句。

C++程序:

```
#include<iostream>
using namespace std;
void main( )
{ char *p;
  try {
    p = new(std::nothrow) char[2000000000]; // p=NULL;
    if (p == NULL)
      throw "Can't get memory!\n";
  }
  catch (char *err)
  {   cout << err; }
}
```

当未能申请到内存，p为空，此时抛出异常，由catch捕获，打印出 Can't get memory!

后可以继续进行程序后面的运行。



直接写 `p = new char[2000000000];` 会导致出现致命错误！
`throw` 没发挥作用。改为：

`p = new(std::nothrow) char[2000000000];`
可以避免 `new` 先抛出错误信息所导致的致命错误。
注意：不同配置的计算机运行结果可能不同。

```
#include<iostream>
using namespace std;
void main( )
{ char *p;
  try {
    p = new(std::nothrow) char[-10]; // p=NULL;
    if (p == NULL)
      throw "Can't get memory!\n";
  }
  catch (char *err)
  {   cout << err; }
```

如果想造成内存申请失败，故意申请-10个字节，能否使p为空，抛出异常，由catch捕获，打印出：

Can't get memory!

呢？编译结果是：

test.cpp(6) : error C2148: 数组的总大小不得超过0x7fffffff 字节

catch在捕获到相应类型的错误值后，还可以根据错误值进一步分类处理打印错误信息，例如：

```
catch (int i)
{
    switch(i)
    {
        case 1001:  cout << "warning....." << endl; break;
        case 1002:  cout << "error....." << endl; break;
        case 1003:  cout << "....." << endl; break;
        case 1004:  cout << "....." << endl; break;
        .....
    }
}
cout << "All done!" << endl;
```

补充：断言assert的使用

```
#include <fstream>
#include <iostream>
#include <cassert> // #include <assert.h>
using namespace std;
#define sz 100
void main( )
{
    char buf[sz];
    ifstream in("strfile.cpp");
    assert(in);
    ofstream out("strfile.out");
    assert(out);
    int i=1;
    while(in.get(buf,sz))
    {
        in.get( );
        cout<<buf<<endl;
        cout<<i++<<": "<<buf<<endl;
    }
}
```

运行结果：Assertion failed: in, file test.cpp, line 10
并弹出错误信息窗，出现致命错误。原因：文件未打开

按F1键获得的assert在线帮助:

- ◆ `assert(CRT)` 评估一个表达式, 当结果为false时, 打印诊断信息并中止程序。
- ◆ `assert`宏通常用于在程序开发过程中识别逻辑错误, 方法是, 仅在程序运行不正确时, 才实现expression参数以评估为false。调试完成后, 断言检查可以通过定义标识符NDEBUG而不用修改源文件来关闭。可以使用/D命令行选项或#define指令定义NDEBUG。如果NDEBUG是用#define定义的, 则指令必须出现在包含ASSERT.H之前。
- ◆ 当表达式计算结果为false(0)并调用中止以终止程序执行时, 断言将打印诊断消息。如果表达式为真(非零), 则不采取任何操作。诊断消息包括失败的表达式, 断言失败的源文件的名称和行号。
- ◆ 在Visual C++ 2005及以后的版本中, 诊断消息以宽字符打印。因此, 即使在表达式中有Unicode字符, 它也会按预期工作。

- ◆ 诊断消息的目标取决于调用例程的应用程序的类型。控制台应用程序始终通过stderr接收消息。在基于Windows的应用程序中，assert调用Windows MessageBox函数来创建一个消息框，以显示该消息以及一个OK按钮。当用户单击确定时，程序立即中止。
- ◆ 当应用程序与运行时库的调试版链接时，assert会创建一个带有三个按钮的消息框：中止，重试和忽略。如果用户单击“中止”，程序立即中止。如果用户单击“重试”，则调用调试器，如果启用了实时（JIT）调试，则用户可以调试该程序。如果用户单击“忽略”，assert会继续正常执行：使用OK按钮创建消息框。请注意，如果在存在错误情况时单击忽略可能会导致未定义的行为。
- ◆ 有关CRT调试的更多信息，请参阅CRT调试技术。
- ◆ 断言例程在C运行时库的发行版和调试版中均可用。其他两个断言宏_ASSERT和_ASSERTE也是可用的，但它们仅在已经定义_DEBUG标志时评估传递给它们的表达式。

```

// crt_assert.c
#include <stdio.h>
#include <assert.h>
#include <string.h>
void analyze_string( char *string ); // Prototype
int main( void )
{ char test1[] = "abc", *test2 = NULL, test3[] = "";
  printf( "Analyzing string '%s'\n", test1 ); fflush( stdout );
  analyze_string( test1 );
  printf( "Analyzing string '%s'\n", test2 ); fflush( stdout );
  analyze_string( test2 );
  printf( "Analyzing string '%s'\n", test3 ); fflush( stdout );
  analyze_string( test3 );
}
// Tests a string to see if it is NULL,
// empty, or longer than 0 characters.
void analyze_string( char * string )
{ assert( string != NULL ); // Cannot be NULL
  assert( *string != '\0' ); // Cannot be empty
  assert( strlen( string ) > 2 ); // Length must exceed 2
}

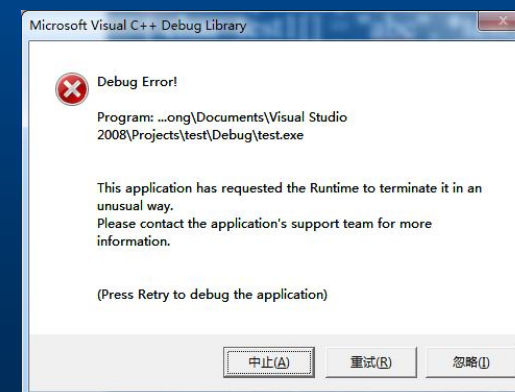
```

运行结果:

Analyzing string 'abc'

Analyzing string '(null)'

Assertion failed: string != NULL, file crt_assert.c, line 24



关于assert（断言）的使用，我从百度文库里找到了一段话：程序一般分为Debug版本和Release版本，Debug版本用于内部调试，Release版本发行给用户使用。断言assert是仅在Debug版本起作用的宏，它用于检查“不应该”发生的情况。以下是一个内存复制程序，在运行过程中，如果assert的参数为假，那么程序就会中止（一般还会出现提示对话，说明在什么地方引发了assert）。

```
//例如，对于复制不重叠的内存块的函数memcpy:  
void memcpy(void *pvTo, void *pvFrom, size_t size)  
{ void *pbTo = (byte *) pvTo;  
  void *pbFrom = (byte *) pvFrom;  
  assert( pvTo != NULL && pvFrom != NULL );  
  while(size-- > 0 )  
    *pbTo++ = *pbFrom++;  
  return (pvTo);  
}
```

assert是一个宏，为了不在程序的Debug版本和Release版本引起差别，assert不应该产生任何副作用。

所以**assert**不是函数，而是宏。程序员可以把**assert**看成一个在任何系统状态下都可以安全使用的无害测试手段。

以下是使用断言的几个原则：

- 1) 使用断言捕捉不应该发生的非法情况。不要混淆非法情况与错误情况之间的区别，后者是必然存在的并且是一定要作出处理的。
- 2) 使用断言对函数的参数进行确认。
- 3) 在编写函数时，要进行反复的考查，并且自问：“我打算做哪些假定？”一旦确定了的假定，就要使用断言对假定进行检查。
- 4) 一般教科书都鼓励程序员们进行防错性的程序设计，但要记住这种编程风格会隐瞒错误。当进行防错性编程时，如果“不可能发生”的事情的确发生了，则要使用断言进行报警。

assert只是一个调试工具，没有太多用处，且比较独立。
而且从`#include <cassert>` 可以看出，这是一个C语言的东西。